

HDOC Day-38 —

Question 1

Variadic Function for Weighted Average

```
#include <stdio.h>
#include <stdarg.h>

double weightedAverage(int count, ...)
{
    va_list args;
    double value, weight;
    double weightedSum = 0.0;
    double totalWeight = 0.0;

    va_start(args, count);

    for (int i = 0; i < count; i++)
    {
        value = va_arg(args, double);
        weight = va_arg(args, double);

        if (weight < 0)
        {
            printf("Error: Weight cannot be negative.\n");
            va_end(args);
            return -1;
        }

        weightedSum = weightedSum + value * weight;
        totalWeight = totalWeight + weight;
    }

    va_end(args);

    if (totalWeight == 0)
    {
        printf("Error: Total weight cannot be zero.\n");
        return -1;
    }

    return weightedSum / totalWeight;
}

int main()
{
    double result;

    result = weightedAverage(3,
                             80.0, 0.3,
                             75.0, 0.2,
                             90.0, 0.5);
}
```

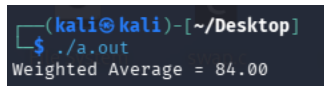
```

    if (result != -1)
    {
        printf("Weighted Average = %.2f\n", result);
    }

    return 0;
}

```

Output



```

(kali@kali)-[~/Desktop]
$ ./a.out
Weighted Average = 84.00

```

Weighted Average = 84.00

Calculation

$[80(0.3)+75(0.2)+90(0.5)] / (0.3+0.2+0.5) = (24+15+45) / 1.0 = 84.00$

Important functions from <stdarg.h>

- `va_list` → stores information about variable arguments.
- `va_start()` → starts accessing variable arguments.
- `va_arg()` → retrieves each argument.
- `va_end()` → ends processing of variable arguments.

Question 2

Command-Line Arguments: Sort and Statistics

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int isInteger(char str[])
{
    int i = 0;

    if (str[0] == '-' || str[0] == '+')
        i = 1;

    if (str[i] == '\0')
        return 0;

    while (str[i] != '\0')
    {
        if (str[i] < '0' || str[i] > '9')

```

```

        return 0;

        i++;
    }

    return 1;
}

void sortArray(int arr[], int n)
{
    int temp;

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

double findAverage(int arr[], int n)
{
    double sum = 0;

    for (int i = 0; i < n; i++)
        sum = sum + arr[i];

    return sum / n;
}

double findMedian(int arr[], int n)
{
    if (n % 2 == 0)
        return (arr[n / 2 - 1] + arr[n / 2]) / 2.0;
    else
        return arr[n / 2];
}

int findSecondLargest(int arr[], int n, int *found)
{
    int largest = arr[n - 1];

    for (int i = n - 2; i >= 0; i--)
    {
        if (arr[i] != largest)
        {
            *found = 1;
            return arr[i];
        }
    }
}

```

```

        *found = 0;
        return 0;
    }

int main(int argc, char *argv[])
{
    int arr[100];
    int n = 0;
    int found;

    for (int i = 1; i < argc; i++)
    {
        if (isInteger(argv[i]))
        {
            arr[n] = atoi(argv[i]);
            n++;
        }
        else
        {
            printf("Invalid argument ignored: %s\n", argv[i]);
        }
    }

    if (n == 0)
    {
        printf("No valid integers entered.\n");
        return 0;
    }

    sortArray(arr, n);

    printf("\nValid integers in ascending order: ");

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");

    printf("Minimum: %d\n", arr[0]);
    printf("Maximum: %d\n", arr[n - 1]);

    printf("Average: %.2f\n",
           findAverage(arr, n));

    printf("Median: %.2f\n",
           findMedian(arr, n));

    int secondLargest = findSecondLargest(arr, n, &found);

    if (found)
        printf("Second-largest distinct value: %d\n", secondLargest);
    else
        printf("Second-largest distinct value does not exist.\n");

    return 0;
}

```

How to compile and run

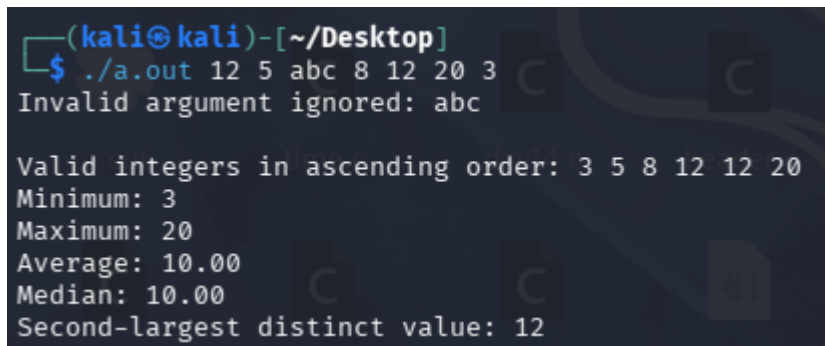
Save the second program as `day38b.c`.

```
cc day38b.c.
```

Run:

```
./a.out 12 5 abc 8 12 20 3
```

Output

A terminal window with a dark background and light blue text. The prompt is `(kali㉿kali)-[~/Desktop]`. The user enters `./a.out 12 5 abc 8 12 20 3`. The program outputs: `Invalid argument ignored: abc`, `Valid integers in ascending order: 3 5 8 12 12 20`, `Minimum: 3`, `Maximum: 20`, `Average: 10.00`, `Median: 10.00`, and `Second-largest distinct value: 12`.

```
(kali㉿kali)-[~/Desktop]
$ ./a.out 12 5 abc 8 12 20 3
Invalid argument ignored: abc

Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12
```

```
Invalid argument ignored: abc
```

```
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12
```